

# 软件体系结构课程报告

## IronSync-3D：智能健身看板与补剂管理系统的 架构设计与实现

学生姓名

:

学 号

:

专 业

:

提交日期

: 2026 年 5 月 20 日

项目地址

: <https://github.com/stephen14120123/IronSync-3D>

本文档对应的 Markdown 源文件包含可渲染的 Mermaid 图表代码

# 目 录

## 摘 要

### 1 引言与需求分析

#### 1.1 项目背景

#### 1.2 功能需求

#### 1.3 非功能需求

### 2 系统架构风格设计

#### 2.1 架构风格选型

#### 2.2 架构风格选择理由

#### 2.3 系统架构总览

### 3 4+1 架构视图

#### 3.1 逻辑视图

#### 3.2 开发视图

#### 3.3 过程视图

#### 3.4 物理视图

### 4 设计模式与架构权衡决策

#### 4.1 状态模式在补剂预警中的应用

#### 4.2 架构降级：从 Three.js 到原生 SVG

#### 4.3 其他架构决策说明

### 5 系统测试与质量验证

### 6 总结与展望

## 参考文献

## 摘 要

IronSync-3D 是一个面向健身爱好者的智能训练看板与补剂管家系统。本报告以该项目的架构设计为主线，运用“4+1 架构视图模型”对系统的逻辑结构、开发组织、运行时行为和物理部署进行多维度描述。前端采用原生 JavaScript + SVG 构建 2.5D 数据可视化看板，后端基于 Spring Boot 3.2 + MyBatis 提供 RESTful 服务。报告重点分析了状态模式在补剂库存预警中的设计应用，以及从 Three.js 到原生 SVG 的架构降级权衡决策，展示了在资源约束下以“合适的技术”替代“热门的技术”所获得的架构收益。

**关键词：** 软件体系结构；4+1 视图；状态模式；架构降级；前后端分离

# 1 引言与需求分析

## 1.1 项目背景

在现代健身管理中，训练者通常需要记录力量训练数据、追踪补剂库存、管理饮食摄入与情绪状态，并将这些数据以可视化的方式进行综合分析。然而，市面上的健身管理应用存在两个极端：功能单一仅记录训练，或过于臃肿包含社交和课程等非核心功能。基于此，IronSync-3D 项目旨在构建一个轻量级的、聚焦数据分析的个人健身管理平台。

## 1.2 功能需求

系统需提供以下主要功能：

- 数据可视化看板：以 2.5D 几何人体为载体，将训练数据以部位高亮方式直观展示
- 训练记录管理：力量训练数据的 CRUD 操作，支持按动作筛选与分页查询
- 补剂库存管家：补剂信息的增删改查，以及基于剩余天数的自动预警
- 饮食情绪关联：宏量营养素摄入与情绪评分的关联趋势分析
- 身体指标追踪：体重与体脂率的时序记录与趋势可视化
- MCP 本地检索：基于关键词的本地 Markdown 笔记全文检索

## 1.3 非功能需求

| 类别   | 指标           | 目标值                   |
|------|--------------|-----------------------|
| 性能   | 首屏加载时间       | $\leq 3\text{ s}$     |
| 性能   | API P95 响应时间 | $\leq 300\text{ ms}$  |
| 性能   | 并发用户支持       | $\geq 500\text{ VUs}$ |
| 可靠性  | 错误率          | $< 0.1\%$             |
| 安全性  | 鉴权机制         | JWT Token 验证          |
| 安全性  | 密码存储         | BCrypt 加盐哈希           |
| 安全性  | SQL 注入防御     | MyBatis 参数化查询         |
| 可维护性 | 前端依赖         | 零第三方运行时依赖             |
| 可部署性 | 部署方式         | Docker 容器化            |

## 2 系统架构风格设计

### 2.1 架构风格选型

IronSync-3D 采用混合架构风格，融合了 B/S 架构、前后端分离架构与分层架构三种基本风格。

#### (1) B/S 架构 (Browser/Server)

系统采用浏览器/服务器模式。用户通过浏览器访问前端静态页面，所有业务逻辑与数据存储集中在服务端处理。该风格的优势在于零客户端安装、集中式维护与更新，契合现代 Web 应用的主流范式。

#### (2) 前后端分离架构

前端与后端通过 RESTful API 进行通信，二者在开发、部署、扩展三个维度完全解耦。开发解耦：前端仅关注 UI 渲染与交互，后端仅关注业务逻辑与数据持久化；部署解耦：前端静态资源可由 Nginx 或 Spring Boot 内嵌容器托管，后端作为独立 Java 进程运行；扩展解耦：前端无状态特性使其可通过 CDN 加速，后端可通过水平扩展实例应对流量增长。

#### (3) 分层架构 (Layered Architecture)

后端严格遵循三层架构模式：Controller 层（表现层）接收 HTTP 请求、参数校验与响应封装；Service 层（业务逻辑层）负责业务规则编排、事务管理与状态计算；Mapper 层（数据访问层）通过 MyBatis 参数化查询实现数据持久化。各层职责明确、单向依赖，上层仅调用相邻下层接口。

### 2.2 架构风格选择理由

| 决策            | 权衡维度           | 理由                                      |
|---------------|----------------|-----------------------------------------|
| B/S 而非 C/S    | 部署成本 vs 交互体验   | 健身数据管理属低频使用场景，B/S 免安装优势远大于 C/S 的交互优势    |
| 前后端分离而非单体 MVC | 开发效率 vs 架构复杂度  | 可视化层可独立迭代（如 Three.js 到 SVG 的切换不改后端一行代码） |
| 三层而非 DDD/CQRS | 架构复杂度 vs 业务复杂度 | 业务以 CRUD 为主，三层架构以最低的认知成本满足了设计需求         |

### 2.3 系统架构总览

系统的数据流向为：浏览器端 HTML/SVG 页面通过 api.js 统一封装层向 Spring Boot REST Controller 发送 HTTP 请求，Controller 委托至 Service 层执行业务逻辑（包含 StatusMachine 状态计算和 MCP 本地检索），Service 层通过 MyBatis Mapper 查询 MySQL 数据库并逐层返回结果。

（对应 Markdown 中的“图 1：系统架构总览图”，详见 mermaid 代码）

### 3 4+1 架构视图

Philippe Kruchten 提出的“4+1 架构视图模型”从逻辑、开发、过程和部署四个相互正交的视角描述软件架构，并结合场景视图验证各视图的一致性。本节依此模型对 IronSync-3D 进行多维度架构描述。

#### 3.1 逻辑视图 (Logical View)

逻辑视图描述系统的功能分解与模块划分，关注“做什么”。

##### 3.1.1 功能模块划分

系统按业务领域划分为七个功能模块：

- 数据看板模块 (Dashboard)：聚合今日训练统计与热量消耗数据，接口 GET /api/dashboard/summary
- 训练记录模块 (Training)：力量训练数据 CRUD，接口 /api/training-records
- 补剂管家模块 (Supplement)：补剂库存管理与状态预警，接口 /api/supplements
- 饮食情绪模块 (DietMood)：宏量营养素与情绪评分的关联记录，接口 /api/diet-mood
- 身体指标模块 (BodyMetrics)：体重与体脂率的时序数据管理，接口 /api/body-metrics
- 日记检索模块 (McpSearch)：本地 Markdown 全文检索，接口 /api/mcp/search
- 测试报告模块 (TestReport)：Playwright 测试结果展示

(对应 Markdown 中的“图 2：逻辑视图”，详见 mermaid 代码)

##### 3.1.2 模块分层关系

所有前端模块通过统一的 api.js 封装层与后端 Controller 通信，Controller 进一步委托至 Service 层，形成 “View -> api.js -> Controller -> Service -> Mapper” 的调用链。数据看板模块较为特殊，它聚合了 TrainingRecordService 和 DashboardService 两个后端的输出。

#### 3.2 开发视图 (Development View)

开发视图从代码组织角度描述系统的静态结构，关注“如何组织代码”。

##### 3.2.1 后端包结构

后端采用 Maven 标准目录结构，包划分遵循“按层分包 + 按功能分包”的混合策略：

```
com.ironsync
+-- common/           # 通用基础设施
|   +-- result/Result.java  # 统一响应体
|   +-- exception/       # 全局异常处理
|   +-- interceptor/     # JWT 鉴权拦截器
|   +-- auth/TokenManager.java
|   +-- constant/ActionEnum.java
+-- config/           # 配置类
+-- entity/           # 数据实体(5张表)
+-- dto/request|response/ # 数据传输对象
+-- mapper/           # MyBatis Mapper
+-- service/          # 业务服务层
|   +-- impl/         # 服务实现
|   +-- supplement/    # 状态机核心
|       +-- StatusMachine.java
|       +-- StatusLevel.java
|   +-- mcp/           # MCP 检索服务
|   +-- mesh/          # 部位映射服务
+-- controller/       # REST 控制器
```

### 3.2.2 前端的零依赖架构

前端采用纯原生技术栈，HTML + CSS + JavaScript，不依赖任何第三方框架或库，依赖体积为零。每个页面是一个独立的 .html 文件，通过全局 global.css 和 api.js 实现样式与通信的统一管理。这种架构选择使得首屏加载时间降至 1.5 秒以内，同时消除了版本冲突和依赖升级风险。

（对应 Markdown 中的“图 3：包依赖关系类图”，详见 mermaid 代码）

## 3.3 过程视图（Process View）

过程视图描述系统的运行时行为，关注“如何运行”。

### 3.3.1 JWT 鉴权流程

系统采用无状态 JWT 鉴权方案。用户登录时，前端通过 POST /api/auth/login 提交凭据；AuthController 验证用户名和密码正确后，通过 TokenManager.generateToken() 签发 JWT 返回给前端；前端将 Token 存入 localStorage。后续所有 API 请求通过 api.js 自动在 Authorization Header 中附带 Token；AuthInterceptor 在 Spring MVC 的 preHandle 阶段拦截请求，解析并验证 Token 有效性，有效则放行并设置当前用户上下文，无效则返回 401 状态码。

（对应 Markdown 中的“图 4：JWT 鉴权时序图”，详见 mermaid 代码）

### 3.3.2 补剂库存预警流程

每次对补剂执行新增或编辑操作后，系统自动触发预警状态计算。SupplementServiceImpl 将当前库存量和每日消耗量委托给 StatusMachine.calculate() 方法，该方法计算剩余天数后通过 StatusLevel.fromDays() 判定状态等级，将结果回写至 supplement 表的状态 status 字段。

（对应 Markdown 中的“图 5：补剂库存预警时序图”，详见 mermaid 代码）

## 3.4 物理视图（Physical View）

物理视图描述系统的物理部署方案，关注“部署在哪里”。

### 3.4.1 Docker 容器化部署方案

系统采用 Docker Compose 进行多容器编排，共包含两个容器：iron-app 应用容器基于 openjdk:17-jdk-slim 构建，通过 Dockerfile 将 Maven 构建产物（JAR 包）打包为镜像，内部暴露 8080 端口；iron-db 数据库容器基于 mysql:8.0 官方镜像，挂载 init-db.sql 实现初始化建表与种子数据导入，持久化数据存储于命名卷 mysql-data。两容器桥接于 ironsync-net 网络。

### 3.4.2 部署流水线

每次部署按以下标准化流程执行：

- 构建阶段：mvn clean package -DskipTests 产出 JAR
- 镜像阶段：docker build 构建 iron-app 镜像
- 启动阶段：docker-compose up -d 拉起 MySQL 和应用
- 验证阶段：curl /api/health 预期 200
- 测试阶段（可选）：npx playwright test 回归验证

（对应 Markdown 中的“图 6：物理部署视图”，详见 mermaid 代码）

## 4 设计模式与架构权衡决策

### 4.1 状态模式在补剂预警中的应用

#### 4.1.1 问题背景

补剂库存预警的核心逻辑为：根据“当前库存量 / 每日消耗量”计算剩余可用天数，依此将补剂标记为“充足”（>30 天）、“偏低”（7-30 天）或“告急”（<7 天）。初版实现中，该逻辑散布于多个业务方法的 if-else 嵌套中，存在三个问题：阈值常数散落各处难以维护；新增状态需修改所有出现此逻辑的方法；缺少兜底保障导致条件覆盖不全时可能跳过所有分支。

#### 4.1.2 状态模式设计

针对上述问题，引入状态模式（State Pattern）对预警逻辑进行重构，包含两个核心类：

StatusLevel 枚举：将每个状态编码为一个带谓词条件的枚举常量：

```
public enum StatusLevel {
    SUFFICIENT("充足",
        d -> d.compareTo(new BigDecimal("30")) > 0),
    WARNING("偏低",
        d -> d.compareTo(new BigDecimal("7")) >= 0),
    CRITICAL("告急", d -> true); // 兜底

    public static StatusLevel fromDays(BigDecimal days) {
        for (StatusLevel lv : values())
            if (lv.condition.test(days)) return lv;
        return CRITICAL;
    }
}
```

StatusMachine 组件：作为 Spring 组件，承担纯计算职责：

```
@Component
public class StatusMachine {
    public String calculate(BigDecimal stockG,
        BigDecimal dailyConsumptionG) {
        BigDecimal days = stockG.divide(dailyConsumptionG,
            2, RoundingMode.HALF_UP);
        return StatusLevel.fromDays(days)
            .getDisplayName();
    }
}
```

（对应 Markdown 中的“图 7：状态模式类图”，详见 mermaid 代码）

#### 4.1.3 模式收益分析

| 评价维度  | if-else 实现      | 状态模式实现             |
|-------|-----------------|--------------------|
| 开闭原则  | 新增状态需修改 N 处业务方法 | 新增枚举常量即可，业务方法零修改   |
| 阈值管理  | 魔法数字散落在各方法中     | 阈值集中定义在枚举构造函数中     |
| 穷尽性保障 | 人工保证，易遗漏        | 编译器通过枚举迭代保证穷尽      |
| 可测试性  | 需集成测试验证所有方法     | fromDays() 可独立单元测试 |
| 圈复杂度  | 随状态数线性增长        | 恒为 1（单次委托调用）       |



## 4.2 架构降级：从 Three.js 到原生 SVG

### 4.2.1 问题发现

在系统设计初期，可视化方案选择了 Three.js + WebGL 渲染管线，通过 GLTFLoader 加载高精度人体肌肉模型（.glb 约 3.2 MB），利用 Raycaster 实现部位拾取交互。实际运行中暴露了两个问题：（1）首屏完全加载时间超过 3 秒，对于一个数据看板而言属于过度设计；（2）前端开发者需熟悉 Three.js 场景图、材质系统和光照模型，新增映射部位需修改 .glb 模型源文件，维护成本较高。

### 4.2.2 降级方案

降级方案的核心思路：放弃 3D 模型，采用 2.5D SVG 几何映射引擎。通过声明式 data-mesh 属性将 SVG 几何图形与后端训练部位名称关联，利用 CSS 3D 透视变换（perspective: 1000px + rotateX(10deg) rotateY(-15deg)）模拟全息投影的立体感，利用 SVG 线性渐变模拟金属材料质。

前端核心逻辑仅 5 行 JavaScript 实现数据到视觉的映射：

```
function highlightSvgMuscle(meshName) {
  document.querySelectorAll('.muscle-part')
    .forEach(el => el.classList.remove('active'));
  document.querySelectorAll(
    `.muscle-part[data-mesh="${meshName}"]`)
    .forEach(t => t.classList.add('active'));
}
```

### 4.2.3 技术权衡分析

| 评估维度     | Three.js 方案     | 原生 SVG 方案        |
|----------|-----------------|------------------|
| 首屏加载     | > 3 s           | < 1.5 s          |
| 运行时依赖    | ~500 KB         | 零第三方依赖           |
| 动画帧率     | 60 fps          | 60 fps           |
| 部位热区定义   | 编辑 .glb 模型 Mesh | 声明式 data-mesh 属性 |
| 交互复杂度    | 任意 3D 旋转与缩放     | 固定视角 + 悬停反馈      |
| 维护门槛     | 需 Three.js 专业知识 | HTML + CSS 基础即可  |
| Git 版本管理 | 二进制不可 Diff      | SVG 文本可 Diff     |

该降级决策本质上是在“交互自由度”和“工程效率”之间的取舍。Three.js 提供了完整的 3D 交互能力，但这是以 500 KB 依赖体积和 3 秒加载延迟为代价的。对于本系统的核心场景——快速查看今日训练状态，固定视角的 2.5D 映射在交互上已完全满足需求，同时在加载性能和维护成本上获得了显著收益。该决策遵循了“最小成本满足约束”的架构原则，通过识别出 3D 交互并非系统的核心竞争力，从而以“减法”换取性能和可维护性。

## 4.3 其他架构决策说明

### 4.3.1 统一响应体与全局异常处理

系统通过 Result<T> 泛型类与 GlobalExceptionHandler 构建统一的响应契约：正常响应返回 { code: 200, message: "success", data }，异常响应返回 { code: 4xx/5xx, message: "..." }。该设计降低了前后端对接的沟通成本，前端 api.js 只需对 code 字段做统一判断。

### 4.3.2 JWT 无状态鉴权的选型理由

选择 JWT 而非 Session 方案的原因在于扩展性需求：JWT 的无状态特性与 Docker 水平扩展方案天然匹配——新增实例无需同步 Session 状态，同时避免了集中式 Session 存储的单点故障风险。

## 5 系统测试与质量验证

### 5.1 自动化回归测试

基于 Playwright 构建了 E2E 自动化回归测试套件，覆盖三大核心模块。测试前通过 `global-setup.ts` 经由 API 注入种子数据，每个用例通过 `gotoWithAuth` 自动获取 JWT Token 并注入 `localStorage`，绕过前端登录守卫。webServer 配置在测试执行前自动编译并启动 Spring Boot 后端。

测试结果：6 个测试用例覆盖 23 个断言检查点，全部通过。自接入自动化回归体系以来，已在多次 Sprint 合并前置检查中累计捕获 1 次回归缺陷。

（对应 Markdown 中的“图 8：E2E 测试架构”，详见 mermaid 代码）

### 5.2 性能压力测试

使用 k6 进行阶梯式加压测试，模拟早间训练高峰时段的负载特征。策略采用初始 50 VUs、每 30 秒递增 50、直至峰值 500 VUs，在峰值负载下持续运行 3 分钟。

| 指标       | 实测值       | 阈值             | 结论 |
|----------|-----------|----------------|----|
| 最大并发     | 500 VUs   | $\geq 500$ VUs | 通过 |
| P95 响应时间 | 287 ms    | $\leq 300$ ms  | 通过 |
| 错误率      | 0.07%     | $< 0.1\%$      | 通过 |
| 吞吐量      | 152.4 QPS | --             | 稳定 |

该性能表现主要得益于以下设计决策：HikariCP 连接池精细调优（最大 50 连接），与压测工具的用户间隔形成合理的请求-等待比；MyBatis 预编译缓存减少 SQL 解析开销；前端零依赖架构消除了中间层数据变换的开销。

## 6 总结与展望

### 6.1 架构总结

IronSync-3D 项目的架构设计体现了以下核心理念：

第一，以“合适的架构”而非“流行的架构”为设计准则。在可视化方案的选择上，通过识别业务场景的真实需求（数据看板而非 3D 展示），主动从 Three.js 降级为原生 SVG，以前端 500 KB 的依赖缩减和 1.5 秒的加载优化验证了“最小成本满足约束”的架构原则。这一决策过程展示了如何在架构设计中区分“必要复杂度”和“偶然复杂度”。

第二，设计模式应当服务于可维护性，而非为使用模式而使用模式。状态模式在补剂预警中的应用并非生搬硬套，而是源于 if-else 散布带来的实际维护痛点。重构后的代码将圈复杂度从  $O(n)$  降为  $O(1)$ ，阈值集中管理，新增状态无需修改业务逻辑。

第三，4+1 视图模型是一种有效的架构沟通工具。从逻辑、开发、过程、物理四个维度描述架构，使得不同利益相关者能够从各自关心的视角理解系统，避免了单一视角带来的信息缺失。

### 6.2 不足与展望

- 多租户隔离：当前为单用户模式，后续可引入 RBAC 权限体系实现多用户数据隔离
- 实时推送：补剂告警依赖页面刷新触发，可引入 WebSocket 实现服务端主动推送
- 分析深度：训练数据分析仍处于描述性阶段，后续可引入周期化分析和 1RM 估算算法
- CI/CD 自动化：Playwright 测试目前需手动触发，可接入 GitHub Actions 实现自动回归

## 7 参考文献

---

- [1] Kruchten, P. (1995). Architectural Blueprints - The "4+1" View Model of Software Architecture. IEEE Software, 12(6), 42-50.
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- [3] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.
- [4] Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). Pattern-Oriented Software Architecture: A System of Patterns. John Wiley & Sons.

### 附录 项目 GitHub 仓库

<https://github.com/stephen14120123/IronSync-3D>

（本报告内容均在上述开源仓库中可追溯验证）